

Як написати власну Inode RAMFS (ATTOFS) з нуля

Цей посібник допоможе вам написати просту файлову систему з підтримкою файлів, папок та інодів. Ми зберігатимемо все у звичайних масивах у пам'яті.

Крок 1: Концепція (Що таке Інод і Запис каталогу)

Уявіть, що файлова система — це гардероб.

1. **Інод (Inode)** — це сама куртка. У ній є кишені (дані), вона має певний розмір (size) і тип (зимова чи літня — файл чи папка). Але на ній немає бірки з іменем.
2. **Запис каталогу (Dirent)** — це номерок, який видає гардеробник. На номерку написано ваше ім'я (name), і він вказує на конкретну куртку (inode_id).

Отже, нам потрібні дві структури і два масиви для них.

Крок 2: Оголошення структур

Створіть файл attofs.h і опишіть ці дві сутності.

1. **Структура Інода (atto_inode_t)** Що має знати про себе файл?

- Тип (Чи це файл, чи папка?). Зробіть константи: 0 (вільно), 1 (файл), 2 (папка).
- Розмір (скільки байтів він займає).
- Вказівник на дані (де в пам'яті лежать самі байти).

2. **Структура Запису Каталогу (atto_dirent_t)** Що має бути написано на "номерку"?

- Ім'я файла (масив символів, наприклад char name[16]).
- Ідентифікатор (ID) інода, на який він вказує (inode_id).
- Ідентифікатор батьківської папки, де лежить цей файл (parent_id). Це дозволить робити структуру /bin/sh.
- Прапорець used (1 — запис зайнятий, 0 — вільний).

Крок 3: Глобальні масиви та Ініціалізація

Створіть файл attofs.c.

1. **Оголосіть масиви:** Створіть два масиви, наприклад, на 16 елементів: один для інодів, інший для записів каталогу.
2. **Функція attofs_init():**
 - Пройдіться циклом по обох масивах і встановіть їх як вільні (для інодів тип 0, для записів used = 0).
 - **Головний трюк:** Створіть кореневу папку / вручну. Вона завжди матиме inode_id = 0.
 - Візьміть inode[0]. Встановіть йому тип "папка" (2).
 - Візьміть dirent[0]. Назвіть його /, used = 1, inode_id = 0, parent_id = 0 (він сам собі батько).

Крок 4: Створення файлів та папок

Напишіть функцію `atofs_create(name, parent_id, type)`. Її логіка проста:

- Пошук вільного місця:**
 - Пройдіться циклом по масиву інодів і знайдіть перший, де тип 0. Запишіть його індекс.
 - Пройдіться циклом по масиву `dirents` і знайдіть перший, де `used == 0`. Запишіть його індекс.
 - Якщо вільного місця немає (один з індексів -1) — поверніть помилку.
- Заповнення даних:**
 - В знайдений інод запишіть `type`, обнулите розмір і вказівник на дані.
 - В знайдений `dirent` запишіть переданий `parent_id`, знайдений індекс інода, встановіть `used = 1` і посимвольно (через цикл `while`) скопіюйте `name`.
- Повернення:** Поверніть знайдений індекс інода.

Крок 5: Пошук файла

Напишіть функцію `atofs_find(name, parent_id)`. Вона потрібна, щоб знайти файл у конкретній папці.

- Пройдіться циклом по всіх `dirents`.
- Якщо `dirent` зайнятий (`used == 1`) і його `parent_id` збігається з тим, який ви шукаєте:
- Порівняйте рядок `name` з іменем у `dirent`. (Напишіть свою просту функцію `strcmp`, яка порівнює символи, поки вони рівні і не зустрінеється `\0`).
- Якщо імена збіглися — поверніть `inode_id` цього запису.

Крок 6: Запис у файл

Напишіть функцію `atofs_write(inode_id, buffer, size)`.

- Перевірка:** Переконайтеся, що `inode_id` валідний і що цей інод має тип "файл" (1).
- Виділення пам'яті:**
 - Викличте свій `kmalloc(size)`.
 - Запишіть отриманий вказівник у `data` вашого інода.
 - Оновіть поле `size` інода.
- Копіювання:**
 - Зробіть цикл `for` від 0 до `size`.
 - Копіюйте байти з `buffer` у виділену пам'ять інода: `inode->data[i] = buffer[i]`.

Крок 7: Читання з файла

Напишіть функцію `atofs_read(inode_id, buffer, size)`.

- Перевірка:** Перевірте `inode_id` і тип. Переконайтеся, що вказівник `data` не дорівнює нулю (тобто файл не порожній).
- Захист від переповнення:** Якщо запитаний `size` більший, ніж розмір файла (`inode->size`), зменшіть `size` до розміру файла.
- Копіювання:**

- Циклом for від 0 до size скопіюйте байти з inode->data у переданий buffer.

Як цим користуватися на практиці

1. Викличте attofs_init().
 2. Створіть папку bin у корені (parent_id 0): `int bin_id = attofs_create("bin", 0, 2);`
 3. Створіть файл sh у папці bin: `int sh_id = attofs_create("sh", bin_id, 1);`
 4. Запишіть дані: `attofs_write(sh_id, "hello", 5);`
 5. Щоб прочитати, спочатку знайдіть інод: `int id = attofs_find("sh", bin_id);`, а потім викличте `attofs_read(id, buf, 5);`.
- Ось і все! Це повноцінна інодна RAMFS, яку ви можете написати самостійно.